



Master in Big Data Analytics

Technological Fundamentals in the Big Data World

Lab 1: K-Means Parallelization in Python

Authors

- Yihang Zhu, S100555169
- Aleksandr Veretelnik, S100578391
- Miguel Rodríguez, S100568756
- Pablo Cano, S100558677

Contents

1	Introduction	3
1.1	Objective	3
2	Generating the proteins	3
3	The algorithm	3
3.1	K-means by scratch	3
3.2	Choosing optimal k (elbow method)	4
3.3	Cluster with the largest average sequence length	5
4	Fitting the proteins	5
4.1	Serial implementation	5
4.2	Parallel processes implementation	6
4.3	Parallel by threads implementation	7
4.4	Plots	8
5	Speed-ups and Amdahl's Law	9
6	Conclusion	10
7	Disclosure on AI usage and references	11

1 Introduction

1.1 Objective

The objective of this lab is to implement, analyze, and optimize the *K-Means* clustering algorithm using Python. The task focuses on applying *K-Means* to a dataset of proteins, where each data-point contains information about the protein's identifier, enzyme type, hydrophobicity, and aminoacid sequence.

The goal is to extract meaningful patterns from this dataset by clustering the proteins based on their Enzyme and Hydrofob fields. The experiment begins with a serial implementation of the algorithm, followed by two parallel versions: one using multiprocessing and the other using threading.

Throughout the lab, we measure and compare execution times across implementations to evaluate the performance improvements obtained through parallelization. The *elbow method* is used to determine the optimal number of clusters, and various visualizations, including cluster plots, centroids, and heat maps, are generated to illustrate the results. Finally, we assess the computational speedup of parallel executions relative to the serial version and estimate the theoretical maximum performance gain using *Amdahl's Law*.

2 Generating the proteins

The dataset used for this lab was generated using the Python script `proteins-generator.py` provided in the lab materials. This script allows the creation of synthetic protein datasets with a specified number of protein chains by executing the command:

```
$ python proteins-generator.py 2000000
```

This command produces a file named `proteins.csv` containing a total of two million data points. Each record in the dataset represents a distinct protein and includes the following features:

- **protid:** a unique identifier assigned to each protein.
- **enzyme:** a categorical variable indicating the enzyme classification of the protein.
- **hydrofob:** a numerical value representing the hydrophobicity of the protein chain.
- **sequence:** a string containing the amino acid sequence of the protein.

To ensure consistency across all experiments, the dataset was generated once and used for every implementation, serial, multiprocessing, and threading, without any modification or additional pre-processing, as required by the laboratory instructions.

3 The algorithm

3.1 K-means by scratch

Let $X = \{x_i\}_{i=1}^n \subset \mathbb{R}^d$ be the data set and let $\{\mu_j\}_{j=1}^k$ denote k centroids with the corresponding clusters $\{C_j\}_{j=1}^k$. The k -means problem seeks to minimize the within-cluster sum of squares (WSS):

$$\min_{\{C_j, \mu_j\}} \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|_2^2.$$

For a fixed partition $\{C_j\}$, the optimal centroid is the arithmetic mean,

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i,$$

Therefore, the algorithm alternates between reassigning points and recomputing means. Given $k \in \mathbb{N}$, the implementation follows the classical K-means iteration:

Initialization. Select k distinct observations as initial centroids $\{\mu_j^{(0)}\}_{j=1}^k$ using a pseudo-random generator for reproducibility.

Assignment step. For each x_i in iteration t , assign it to its nearest centroid according to the Euclidean distance:

$$c_i^{(t)} = \arg \min_{1 \leq j \leq k} d(x_i, \mu_j^{(t)}) = \arg \min_{1 \leq j \leq k} \sqrt{(\mu_j^{(t)} - x_i)^2}.$$

Update step. Recompute each centroid as the mean of its assigned points:

$$\mu_j^{(t+1)} = \frac{1}{|\{i : c_i^{(t)} = j\}|} \sum_{i: c_i^{(t)} = j} x_i.$$

If a cluster becomes empty, the implementation reseeds its centroid to a random data point to prevent degeneration and allow subsequent iterations to proceed.

Convergence criterion. Let $M^{(t)} \in \mathbb{R}^{k \times d}$ stack the centroids row-wise at iteration t . The loop terminates when the absolute shift in centroids falls below a tolerance,

$$\|M^{(t+1)} - M^{(t)}\|_F < \text{tol},$$

or when a maximum number of iterations is reached. Here, $\|\cdot\|_F$ denotes the Frobenius norm.

WSS computation The k -means inertia (WSS) is

$$\text{WSS} = \sum_{j=1}^k \sum_{i: c_i=j} \|x_i - \mu_j\|_2^2.$$

In the implementation, WSS is computed directly after convergence by summing the squared deviations of each point from its assigned centroid:

$$\widehat{\text{WSS}} = \sum_{j=1}^k \sum_{i: c_i=j} (x_i - \mu_j)^\top (x_i - \mu_j).$$

In summary, the code performs vectorized Euclidean assignments, mean-based updates with empty-cluster reseeding, an absolute centroid-shift stopping rule, and reproducible seeding via a fixed random generator to implement the K-means. Finally, it calculates and records WSS for each k to prepare for the search for an optimal k .

3.2 Choosing optimal k (elbow method)

Let the candidate set be $\mathcal{K} = \{k_1, \dots, k_m\}$ with the corresponding WSS values $y_r = \text{WSS}(k_r)$. Define the endpoints $P_1 = (k_1, y_1)$ and $P_2 = (k_m, y_m)$ and the line between two extreme points $\ell = P_2 - P_1$. For each point $P = (k_r, y_r)$, the perpendicular distance to the line is

$$\text{dist}(P, \ell) = \frac{|\ell_x(y_r - y_1) - \ell_y(k_r - k_1)|}{\sqrt{\ell_x^2 + \ell_y^2}} = \frac{|\ell \times (P - P_1)|}{\|\ell\|_2}.$$

The selected k^* is the one that maximizes (3.2):

$$k^* = \arg \max_{k_r \in \mathcal{K}} \text{dist}((k_r, \text{WSS}(k_r)), \ell).$$

A small ε is added to the denominator in code to avoid division by zero.

3.3 Cluster with the largest average sequence length

Let $c_i \in \{1, \dots, k\}$ be the cluster label of sample i and let $L_i \in \mathbb{N}$ denote its sequence length (number of amino acids). For cluster j , define the index set $S_j = \{i : c_i = j\}$ and $n_j = |S_j|$. The average sequence length of cluster j is

$$\bar{L}_j = \begin{cases} \frac{1}{n_j} \sum_{i \in S_j} L_i, & \text{if } n_j > 0, \\ -\infty, & \text{if } n_j = 0, \end{cases}$$

where assigning $-\infty$ to empty clusters ensures they cannot be selected. The reported cluster is the maximizer

$$j^* = \arg \max_{j \in \{1, \dots, k\}} \bar{L}_j, \quad \text{with summary } (j^*, \bar{L}_{j^*}, n_{j^*}).$$

4 Fitting the proteins

4.1 Serial implementation

Code Logic

1. **Load & features.** Read `proteins.csv`; extract two numeric columns (`enzyme`, `hydrofob`) as a feature matrix $\text{features} \in \mathbb{R}^{n \times 2}$. Compute sequence lengths $\text{lengths} \in \mathbb{N}^n$ from the string column `sequence`.
2. **K-means**
 - (a) Generate a list of k from 1 to 15 as possible optimal number of clusters that are fed to the `kmeans` function
 - (b) For each clustering, pick k distinct rows as initial centroids using a seeded NumPy RNG for reproducibility.
 - (c) Compute the distance of each data point to the existing centroids, classify data points to the nearest cluster, update the centroid position and repeat the above process until the changes of the centroid positions are negligible after certain times of iteration as shown in section 3.1.
 - (d) After convergence, compute WSS
3. **Choose k (elbow).** Run K-means for $k = 1, \dots, 15$, store each run's WSS, then pick k_* by the *maximum distance-to-chord* rule, i.e. selecting the k that maximizes the distance between the k point and the line between two farthest k points in the elbow graph.
4. **Final clustering.** Refit K-means once at k_* to obtain final labels and centroids as well as compute the average sequence length of each cluster and report the cluster with the highest average length value.
5. **Visualization.**
 - *Elbow plot:* WSS vs. k to find the optimal number of clusters and show diminishing returns.

- *Scatter plot*: points in (`enzyme`,`hydrofob`) colored by cluster; centroids marked with large “X”.
 - *Heat map*: $2 \times k_*$ matrix of centroid coordinates with numeric annotations.
6. **Timing & output.** Print k_* , the longest-sequence cluster summary, the elbow sweep time, and the total runtime; finally render all figures.

Results

In the serial implementation, the K-Means algorithm was executed using a single process. The elbow Method identified an optimal number of clusters of $k = 3$. The cluster with the highest average sequence length was cluster number 2, with an average sequence length of approximately 126.06 amino acids.

Regarding performance, the average time to compute the elbow curve was **44.0 s**, and the total execution time was approximately **46.0 s**. This version establishes the baseline for subsequent performance comparisons.

4.2 Parallel processes implementation

Code logic

1. **Load & features.** The same input loading and feature extraction steps are performed as in the serial implementation.
2. **Parallel elbow computation.**
 - (a) Instead of running the K-Means algorithm sequentially for each k , a list of candidate values $k = 1, \dots, 15$ is distributed among multiple processes using the `multiprocessing.Pool` API with the `spawn` start method.
 - (b) Each worker process executes the helper function `elbow_worker()`, which performs a full K-Means fit for a given k and returns its inertia (WSS).
 - (c) The main process collects all results using `pool.starmap()` and sorts them by k to reconstruct the complete Elbow curve.
3. **K-means clustering.** After determining the optimal k_* using the same distance-to-chord elbow heuristic, the K-Means algorithm is run once more (serially) to compute the final cluster assignments and centroids.
4. **Cluster statistics.** Compute the average amino acid sequence length per cluster and identify the cluster with the largest average length, as in the serial implementation.
5. **Visualization.**
 - *Elbow plot (MP)*: Displays the WSS vs. k curve computed in parallel across processes.
 - *Scatter plot*: Clustering result in the (`enzyme`,`hydrofob`) space with centroids marked.
 - *Heat map*: $2 \times k_*$ matrix showing the centroid values.
6. **Timing & output.** Print the optimal k_* , the best cluster summary, the elapsed time for the parallel Elbow computation, and the total runtime. The multiprocessing version replaces the sequential loop over k with true process-level parallelism, reducing computation time substantially.

Results

The multiprocessing version of the K-Means implementation achieved a substantial performance improvement. As in the serial case, the optimal number of clusters obtained through the elbow Method was $k = 3$, and the cluster with the highest average sequence length corresponded to cluster number 1, with an average length of 126.07 amino acids.

The average time to compute the elbow curve in the parallelized portion was **17.6 s**, while the total execution time of the program was reduced to approximately **20.1 s**. This represents a significant reduction compared to the serial execution, demonstrating the efficiency gained from distributing the workload across multiple processes.

4.3 Parallel by threads implementation

Code logic

1. **Load & features.** The same input loading and feature extraction steps are performed as in the serial implementation.
2. **Parallel elbow computation (threads).**
 - (a) The elbow computation is parallelized using the `threading` module instead of multiprocessing.
 - (b) A `queue.Queue` structure is used to distribute the list of k values ($1, \dots, 15$) among worker threads.
 - (c) Each thread repeatedly takes a k from the queue, executes K-Means on the shared data, and stores its resulting inertia in a shared output queue.
 - (d) Once all tasks are completed, the threads receive a `None` sentinel to terminate safely. The main thread then aggregates and sorts the results by k .
3. **K-means clustering.** The optimal k_* is again determined by the elbow method. A final serial K-Means run produces the cluster labels and centroids.
4. **Cluster statistics.** Compute the mean sequence length per cluster and identify the one with the highest average value.
5. **Visualization.**
 - *Elbow plot (TH)*: Shows WSS vs. k computed via concurrent threads.
 - *Scatter plot*: Displays cluster assignments with centroid positions.
 - *Heat map*: Displays centroid feature values, identical in format to other versions.
6. **Timing & output.** Print timing information for the threaded elbow computation and total runtime. Although threading enables concurrent execution, the Python Global Interpreter Lock (GIL) prevents true CPU parallelism, resulting in smaller speed-ups compared to the multiprocessing version.

Results

In the multithreading implementation, the results of the clustering were consistent with previous versions. The optimal number of clusters was again $k = 3$, and the cluster with the highest average sequence length was cluster number 1, with an average of 126.07 amino acids.

Execution-wise, the average elbow computation time was **20.4 s**, and the total runtime reached approximately **22.9 s**. Although threading improved the performance compared to the serial implementation, it was slightly slower than the multiprocessing version.

4.4 Plots

Since the same dataset was consistently applied across all implementations, the resulting elbow curves, cluster distributions, and centroid heat maps did not display any significant visual differences. Nevertheless, in order to provide a complete comparison of the methodologies, the corresponding graphs for the three execution modes (Serial, Multiprocessing, and Threading) are presented below.

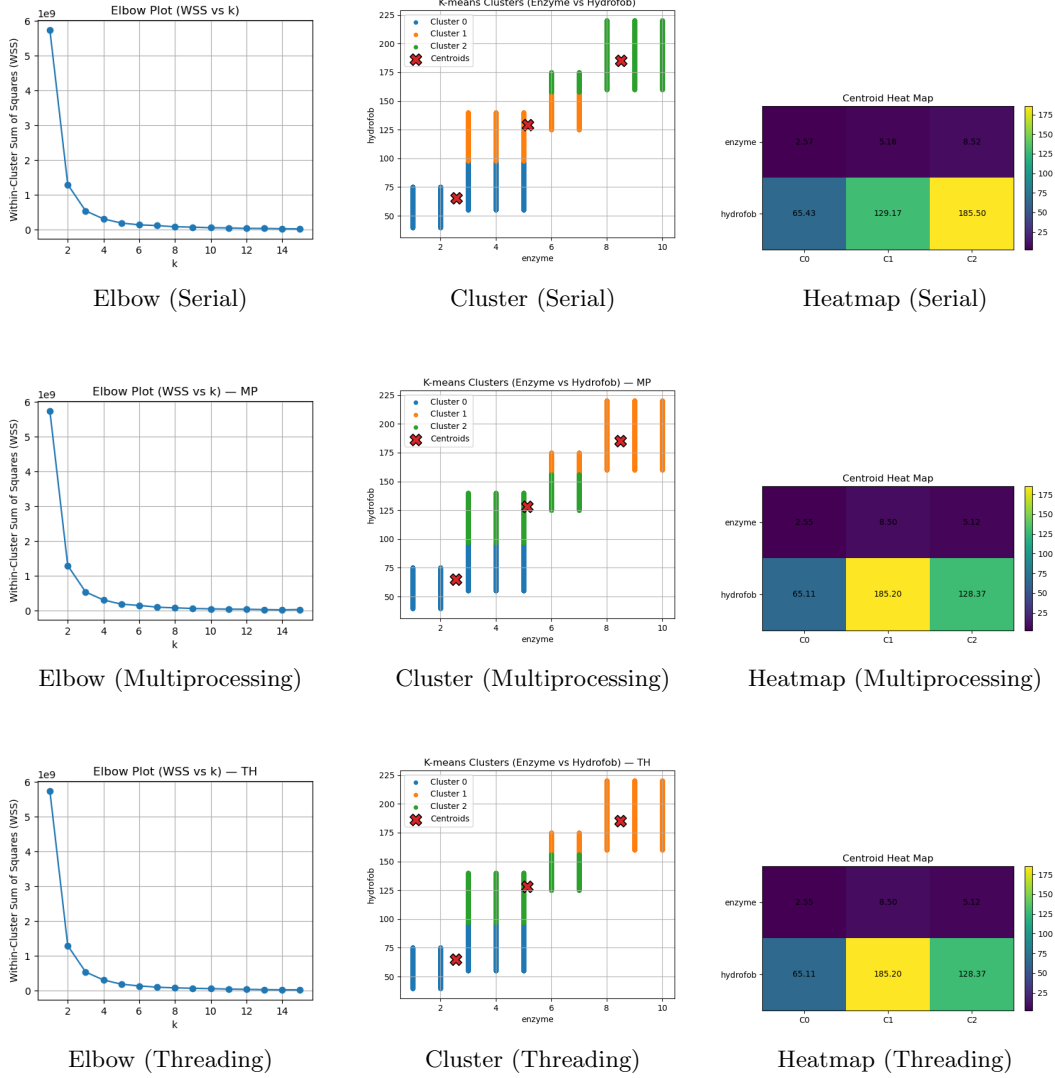


Figure 1: Comparative K-Means results in Serial, Multiprocessing, and Threading: elbow curves, cluster distributions, and centroid heatmaps.

5 Speed-ups and Amdahl's Law

Speed-ups comparison

To ensure reliability of performance measurements, each version was executed five times, and the averages were used to compute speed-up factors. Table 1 summarizes the mean execution times for the serial, multiprocessing, and threading implementations, separating the elbow computation (parallel section) from the total program time.

Implementation	Elbow time (s)	Total time (s)	Elbow Speed-up	Total Speed-up
Serial	44.02	46.05	1.00	1.00
Multiprocessing	17.63	20.10	2.50	2.29
Threading	20.45	22.91	2.15	2.01

Table 1: Average execution times and speed-ups for each implementation.

The multiprocessing version achieved the highest performance gain, reaching a speed-up of approximately $2.5\times$ for the parallel portion (elbow computation) and $2.3\times$ overall. The threading version also improved over the serial baseline but remained less efficient due to Python's thread execution model.

Amdahl's Law

Amdahl's Law estimates the theoretical limit of speed-up based on the fraction of code that can be parallelized. It is given by:

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

where S is the overall speed-up, P is the parallel fraction of the code, and N the number of available cores.

To calculate the fraction of the code that is parallelizable, we measure the execution time of the entire code, denoted as t_s . Then we measure the execution time of the part of the code that is going to be parallelized (the time is measured in the serial implementation), denoted as t_p . The fraction of the code that is parallelizable is then given by

$$P = \frac{t_p}{t_s}.$$

Given the data $t_s = 46.05$ and $t_p = 44.02$, we obtain

$$P = \frac{44.02}{46.05} = 0.956.$$

Below is a list of the theoretical speedup for different numbers of cores:

$$\begin{aligned} N = 8, \quad S_8 &= \frac{1}{(1 - 0.956) + \frac{0.956}{8}} = 5.52, \\ N = 10, \quad S_{10} &= \frac{1}{(1 - 0.956) + \frac{0.956}{10}} = 7.17, \\ N = 16, \quad S_{16} &= \frac{1}{(1 - 0.956) + \frac{0.956}{16}} = 7.69, \\ N \rightarrow \infty, \quad S_\infty &= \frac{1}{1 - 0.956} = 22.73. \end{aligned}$$

This indicates that with unlimited parallel resources, the program would have a speedup of 22.73.

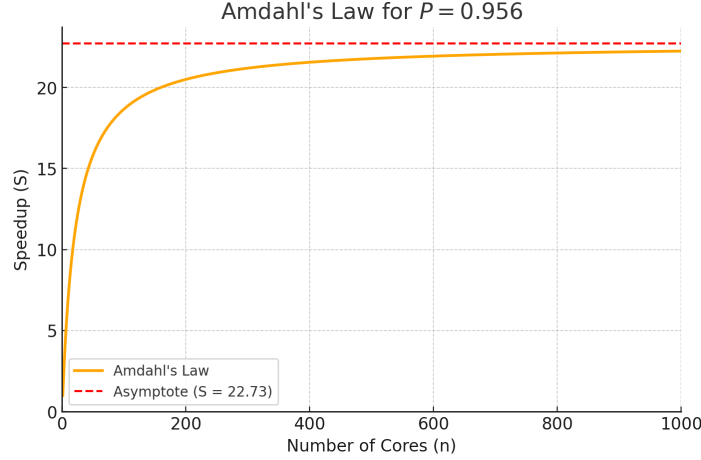


Figure 2: Amdahl’s Law for $P = 0.956$. The orange curve shows the theoretical speedup as the number of cores increases, and the dashed red line represents the asymptotic limit $S_{\infty} = \frac{1}{1-P} = 22.73$.

6 Conclusion

This laboratory provided a practical exploration of the K-Means clustering algorithm and its performance under different execution models: serial, multiprocessing, and multithreading. By applying K-Means to a dataset of two million synthetic protein records, we analyzed how parallelization strategies influence computational efficiency in Python for CPU-bound tasks.

All implementations consistently identified three optimal clusters ($k = 3$) using the *elbow method*, with negligible differences in clustering outcomes. However, clear differences emerged in computational performance across execution models.

The multiprocessing version achieved the best results, reducing the total execution time from approximately 46 seconds in the serial case to about 20 seconds, corresponding to an overall speed-up of roughly $2.3\times$. The multithreading implementation also improved over the serial baseline but achieved a smaller acceleration ($2.0\times$) due to the limitations imposed by Python’s Global Interpreter Lock (GIL), which restricts true parallelism in CPU-bound workloads.

Using Amdahl’s Law, the parallelizable fraction of the code was estimated at $P = 0.956$, indicating that about 95.6% of the program benefits from parallelization. The theoretical analysis predicted a maximum speed-up of $S_{\infty} = 22.73\times$, showing that further gains are possible with more efficient multi-core utilization or lower inter-process overhead. The measured performance aligns with these theoretical expectations, validating both the correctness and efficiency of the multiprocessing implementation.

Overall, the experiment achieved its main objectives:

- Implemented the K-Means algorithm from scratch and validated it on a large dataset.
- Applied and compared serial, multiprocessing, and threading approaches to assess their efficiency.
- Quantified empirical and theoretical speed-ups using Amdahl’s Law.
- Demonstrated that multiprocessing offers the most effective parallelization strategy for CPU-bound K-Means workloads in Python.

7 Disclosure on AI usage and references

AI tools, including ChatGPT by OpenAI, were used throughout this project to assist in rewriting and refining text, identifying and correcting coding errors, and clarifying concepts related to K-Means parallelization. All AI-generated content and code suggestions were carefully reviewed, tested, and verified by the authors to ensure their accuracy and validity within the context of the laboratory work.

References

1. Maimon, O., & Rokach, L. (2006). *Data mining and knowledge discovery handbook*. Springer.
https://doi.org/10.1007/978-0-387-30164-8_425

Appendix

Execution Time Measurements

This appendix presents the raw execution times obtained from running each version of the program five times. The values are separated into *elbow time* (the parallelizable portion) and *Total time* (complete program execution). Averages were computed to minimize the effect of system and scheduling variability.

Implementation	Run 1	Run 2	Run 3	Run 4	Run 5	Average
Elbow Time (s)						
Serial	43.801	44.217	43.963	44.196	43.938	44.02
Multiprocessing	15.335	18.454	20.081	18.812	15.482	17.63
Threading	23.572	17.537	17.113	25.570	16.457	20.45
Total Time (s)						
Serial	45.817	46.228	45.978	46.227	45.996	46.05
Multiprocessing	17.770	20.922	22.540	21.290	17.986	20.10
Threading	26.041	19.997	19.574	28.029	18.919	22.91

Table 2: Raw execution times (in seconds) for Serial, Multiprocessing, and Threading implementations.

All executions were made in the same computer (Macbook Air, Chip M4, 24GB memory) and in similar conditions (only Visual Studio Code running).